

TRACEFORGE V2 — SECURITY AUDIT VERDICT

Audit Date: June 6–7, 2026

Auditor: PKS (theconsoler)

Tools Used: Semgrep OSS (auto + p/python), Bandit, pip-audit, Safety, detect-secrets

Manual Review: Phase 3 — 6 checks across all critical code paths

Codebase: 3,104 lines across 31 git-tracked files

Python Version: 3.10.12



Executive Summary

Tool Results at a Glance

Tool	Scope	Findings
Semgrep OSS (auto)	31 files, 339 rules	6 blocking
Semgrep OSS (p/python)	20 files, 151 rules	2 blocking (overlap)
Bandit	3,104 lines	1 HIGH, 89 LOW
pip-audit	10 packages	30 CVEs
Safety	181 packages, 20 affected	64 vulnerabilities
detect-secrets	Full codebase, 27 detectors	0 secrets found ✓

Code-Level Findings Summary

Metric	Value
Unique code findings	6
Critical / HIGH	2
Medium	2
Fixed during audit	1
False positives closed	1
Files affected	3

Manual Review Summary (Phase 3)

Check	Result
SQL Injection (ledger.py + store.py)	✓ Clean — all parameterized queries
Path Traversal (Flask routes)	✓ Clean — whitelist validation exists
CSRF Protection	✓ Fixed — CSRFProtect applied during audit
Command Injection (subprocess)	✓ Clean — all list-form, no shell=True
Pickle Deserialization	✓ Clean — zero pickle usage in project code
Scapy PCAP file validation	✓ Clean — path.exists() check before rdpcap()
Hardcoded Secrets (grep)	✓ Clean — zero secrets found
Git history secrets	✓ Clean — nothing in history
.env in .gitignore	✓ Confirmed

Dependency Findings Summary

Metric	Value
Vulnerable packages (pip-audit)	10 packages / 30 CVEs
Vulnerable packages (Safety)	20 packages / 64 CVEs
Most critical dependency	gitpython 3.1.24 — RCE via crafted repo
Hardcoded secrets	0 ✓
Git history secrets	0 ✓

Overall verdict: HIGH RISK on dependencies, LOW-MODERATE RISK on code.

The codebase is cleaner than expected — manual review found zero exploitable issues

beyond what automated tools caught. The dependency layer is the real problem

—
64 CVEs across 20 outdated packages including RCE-class vulnerabilities.

Findings Log

Finding #1

Field	Detail
Severity	HIGH
File	dashboard/app.py

Field	Detail
Line	158
Vulnerability Class	Remote Code Execution via Flask Debug Mode
Tool	Semgrep + Bandit (independently confirmed)
Rule	<code>python.flask.security.audit.debug-enabled</code> / B201
Status	✗ OPEN

Vulnerable Code:

```
app.run(debug=True, host="0.0.0.0", port=5000)
```

Why it's critical:

When `debug=True` is active, the Werkzeug interactive debugger is exposed. Any unhandled exception shows a browser-based Python console — an attacker can execute arbitrary Python code on the server with zero authentication. This is textbook Remote Code Execution (RCE).

Fix:

```
import os
app.run(
    debug=os.getenv("FLASK_DEBUG", "false").lower() == "true",
    host=os.getenv("FLASK_HOST", "127.0.0.1"),
    port=int(os.getenv("FLASK_PORT", 5000))
)
```

Add to `.env`:

```
FLASK_DEBUG=false
FLASK_HOST=127.0.0.1
FLASK_PORT=5000
```

Finding #2

Field	Detail
Severity	HIGH

Field	Detail
File	dashboard/app.py
Line	158
Vulnerability Class	Server Exposed on All Network Interfaces
Tool	Semgrep
Rule	python.flask.security.audit.app-run-param-config
Status	✗ OPEN

Vulnerable Code:

```
app.run(debug=True, host="0.0.0.0", port=5000)
```

Why it's critical: `host="0.0.0.0"` binds the Flask server to every network interface.

In a VM or cloud environment the dashboard is reachable from any machine on the network. Combined with `debug=True` — fully exposed RCE over network.

Fix: Same as Finding #1 — one fix resolves both.

Finding #1 and #2 are the same line. One fix kills both.

Finding #3

Field	Detail
Severity	MEDIUM
File	Dockerfile
Lines	46, 47
Vulnerability Class	Container Running as Root
Tool	Semgrep
Rule	dockerfile.security.missing-user-entrypoint + missing-user
Status	✗ OPEN

Vulnerable Code:

```
ENTRYPOINT ["python3.11", "-m", "traceforge.cli"]
CMD ["--help"]
# No USER instruction before these lines
```

Why it matters:

Without a `USER` directive every process inside the container runs as root. If an attacker achieves code execution inside the container they immediately have root privileges — container escape or full data exfiltration.

Fix:

```
RUN groupadd -r traceforge && useradd -r -g traceforge traceforge
USER traceforge

ENTRYPOINT ["python3.11", "-m", "traceforge.cli"]
CMD ["--help"]
```

Finding #4

Field	Detail
Severity	MEDIUM
File	<code>traceforge/core/report.py</code>
Lines	267–276
Vulnerability Class	Direct Jinja2 Usage — Potential XSS
Tool	Semgrep
Rule	<code>python.flask.security.xss.audit.direct-use-of-jinja2</code>
Status	⚠️ NEEDS MANUAL CHECK

Flagged Code:

```
html_content = template.render(
    case=case,
    generated_at=generated_at,
    total_artifacts=total_artifacts,
    evidence=evidence,
    correlations=correlation_dicts,
)
```

Why it matters:

Direct Jinja2 rendering bypasses Flask's automatic HTML autoescaping.

If any variable contains unsanitized user-controlled HTML it renders raw — XSS.

Manual check:

```
grep -n "autoescape\|Environment" traceforge/core/report.py
```

Fix:

```
from jinja2 import Environment, FileSystemLoader
env = Environment(
    loader=FileSystemLoader("templates"),
    autoescape=True
)
template = env.get_template("report.html")
```

Finding #5 — FIXED

Field	Detail
Severity	MEDIUM
File	dashboard/app.py
Vulnerability Class	Missing CSRF Protection
Tool	Manual grep — no CSRFProtect found
Status	 FIXED during audit

Issue:

Zero CSRF protection on Flask POST routes. Any form submission could be forged by a malicious third-party site (Cross-Site Request Forgery).

Fix Applied:

```
# Added to dashboard/app.py
from flask_wtf.csrf import CSRFProtect # line 8
app.secret_key = os.getenv("SECRET_KEY", "change-this-in-pr
oduction") # line 24
csrf = CSRFProtect(app) # line 25
```

Verified:

```
grep -n "secret_key\|CSRFProtect" ./dashboard/app.py
8:  from flask_wtf.csrf import CSRFProtect
24: app.secret_key = os.getenv("SECRET_KEY", "change-this-in-production")
25: csrf = CSRFProtect(app)
```

Finding #6 — FALSE POSITIVE CLOSED

Field	Detail
Severity	MEDIUM (flagged) → CLOSED
File	<code>traceforge/cli.py</code>
Line	292
Vulnerability Class	Dynamic Module Import
Tool	Semgrep
Status	 FALSE POSITIVE — CLOSED

Flagged Code:

```
mod = importlib.import_module(mod_path)
```

Manual verification result: `mod_path` is constructed internally from CLI subcommand routing — not from raw user input. The module path is controlled by Click's command dispatch system, not passed directly from untrusted user data. No whitelist needed.

INFORMATIONAL

Finding I1 — 89 Low Severity Bandit Findings

Mostly `assert` statements, `hashlib.md5()` for file hashing (not passwords), and broad `except` clauses. Not exploitable in isolation. Revisit after all HIGH/MEDIUM findings are resolved.

Finding I2 — Semgrep Pro Rules Not Scanned

1,859 advanced rules missed without a Semgrep account. Create a free account

at <https://sg.run/rules> then run:

```
semgrep login
semgrep --config=auto . --exclude-dir=venv --output semgrep
_report_pro.txt
```

Dependency Audit

pip-audit — 30 CVEs Across 10 Packages

Package	Version	CVEs	Fix Version	Risk
gitpython	3.1.24	8	3.1.50	 RCE via crafted repo
pillow	9.0.1	8	10.3.0	 Image parsing RCE
pyjwt	2.12.1	3	2.13.0	 Algorithm confusion
idna	3.3	3	3.7	 DoS via crafted domain
mako	1.1.3	2	1.3.12	 XSS + path traversal
paramiko	2.9.3	2	3.4.0	 Auth bypass
oauthlib	3.2.0	1	3.4.0	 DoS
future	0.18.2	1	1.0.0	 DoS
reportlab	3.6.8	1	3.6.13	 PDF parsing
pyml	1.5.0	1	—	 Low severity

Safety — 64 CVEs Across 20 Packages

Safety found 10 additional packages pip-audit missed:

Package	Notable CVE	Class
pip	CVE-2023-5752, CVE-2026-1703	Command injection, path traversal
protobuf	CVE-2026-0994, CVE-2022-1941	DoS, message parsing
pynacl	CVE-2025-69277	Cryptography misuse
bandit	PVE-2024-64484	SQL injection false negative risk
+ 6 more	Various	Low-medium severity

Safety `check` command is deprecated. Use `safety scan` in future runs.

Top 3 That Actually Matter for TraceForge

1. gitpython 3.1.24 — TraceForge does git history analysis. A malicious repo passed to TraceForge could trigger RCE. Fix: `pip install --upgrade gitpython`

2. pillow 9.0.1 — If any forensic images are processed, a malformed file triggers RCE. 8 CVEs. Fix: `pip install --upgrade pillow`

3. pip 22.0.2 — pip itself is vulnerable to command injection via Mercurial VCS URLs.

Fix: `pip install --upgrade pip`



Secrets Audit — detect-secrets

Detector	Result
AWS Key Detector	✓ Clean
GitHub/GitLab Token Detector	✓ Clean
JWT Token Detector	✓ Clean
Private Key Detector	✓ Clean
High Entropy String (Base64/Hex)	✓ Clean
Basic Auth Detector	✓ Clean
21 other detectors	✓ Clean
Git history secrets	✓ Clean
.env in .gitignore	✓ Confirmed
Hardcoded secrets grep	✓ Clean

Zero hardcoded secrets anywhere in the codebase.



Fix Priority Checklist

CODE FIXES — do in this order

[] #1 + #2 — dashboard/app.py line 158
debug=False, host=127.0.0.1, move to .env

[] #3 — Dockerfile lines 46-47
Add USER nonroot before ENTRYPOINT

- [] #4 – traceforge/core/report.py lines 267-276
Verify autoescape=True in Jinja2 Environment
Fix if not set
- [] #5 – CSRF ☒ ALREADY FIXED
CSRFProtect applied during audit
- [] #6 – cli.py line 292 ☒ FALSE POSITIVE CLOSED
No action needed

DEPENDENCY FIXES

- [] DEP-1 – pip install --upgrade gitpython
8 CVEs including RCE
- [] DEP-2 – pip install --upgrade pillow
8 CVEs including RCE
- [] DEP-3 – pip install --upgrade pip
6 CVEs including command injection
- [] DEP-4 – pip install --upgrade pyjwt idna mako paramiko oauthlib
Medium severity CVEs
- [] DEP-5 – pip install --upgrade protobuf
4 CVEs including DoS
- [] DEP-6 – pip freeze > requirements.txt
Update after all upgrades
- [] DEP-7 – pip-audit && safety scan
Confirm zero findings after upgrades

SEMGREP

- [] OPT-1 – semgrep login (free account)
Unlock 1859 pro rules
Re-run: semgrep --config=auto . --exclude-dir=venv



Phase 5 — Fix Priority Reference

Priority	Vulnerability Class	Fix
P1	Flask debug=True + 0.0.0.0 (RCE)	<code>.env</code> controlled config
P1	Dependency RCEs (gitpython, pillow)	<code>pip install --upgrade</code>
P2	Docker runs as root	Add <code>USER nonroot</code> to Dockerfile
P2	Jinja2 autoescape not confirmed	Enable <code>autoescape=True</code>

Priority	Vulnerability Class	Fix
🟡 P2	Remaining dependency CVEs	Upgrade all flagged packages
🟢 P3	CSRF	✅ Already fixed
🟢 P3	Semgrep pro rules not run	Create free account + re-scan

Glossary — Security Terms Used in This Audit

RCE — Remote Code Execution

The ability for an attacker to run arbitrary code on your server without physical access.

The most severe vulnerability class. Finding #1 (debug=True) is an RCE.

An attacker who triggers any Python exception on your Flask app gets an interactive

Python console in their browser — they can run `os.system("rm -rf /")` from their laptop.

CVE — Common Vulnerabilities and Exposures

A standardized ID assigned to a publicly known vulnerability.

Format: CVE-YEAR-NUMBER (e.g. CVE-2022-24439).

Maintained by MITRE, published in the National Vulnerability Database (NVD).

Every CVE has a CVSS score rating its severity 0–10.

CVSS — Common Vulnerability Scoring System

A numerical score (0–10) that rates vulnerability severity.

0–3.9 = Low, 4–6.9 = Medium, 7–8.9 = High, 9–10 = Critical.

The CVEs in gitpython and pillow score 8.7+ — that's High severity.

SQL Injection

Inserting malicious SQL into a query via user input to manipulate the database.

Example — if a query is built as `"SELECT * FROM users WHERE id=" + user_input`, an attacker passes `1 OR 1=1` to dump the entire table.

TraceForge is clean — all queries use parameterized `?` placeholders.

Command Injection

Inserting shell commands into a system call via user input.

`subprocess.call("ping " + user_input, shell=True)` — attacker passes

`8.8.8.8; rm -rf /` and both commands execute.

TraceForge is clean — all subprocess calls use list form without `shell=True`.

Path Traversal

Using `../` sequences in file paths to escape an intended directory.

`send_file('/uploads/' + filename)` — attacker passes `../../../../etc/passwd`.

TraceForge is clean — `fmt` parameter is whitelisted before `send_file()`.

CSRF — Cross-Site Request Forgery

Tricking a logged-in user's browser into making unauthorized requests to your app.

A malicious site can submit a form to your Flask app using the victim's session. Fixed in TraceForge by adding `CSRFProtect` which validates a hidden token on every POST.

XSS — Cross-Site Scripting

Injecting malicious JavaScript into a web page viewed by other users.

If `<script>alert(1)</script>` is stored as a case name and rendered without escaping — it executes in every viewer's browser.

Prevented by Jinja2 autoescaping (Finding #4 — needs verification).

Pickle Deserialization

Python's pickle format executes arbitrary code during loading by design.

A malicious `.pkl` file can run any system command when loaded with

`pickle.load()`.

TraceForge has zero pickle usage — clean.

SAST — Static Application Security Testing

Analyzing source code for vulnerabilities without running it.

Semgrep and Bandit are SAST tools — they read your code and flag dangerous patterns.

SCA — Software Composition Analysis

Scanning your dependencies for known vulnerabilities.

`pip-audit` and `Safety` are SCA tools — they check your `requirements.txt` against CVE databases and flag outdated packages.

CNA — CVE Numbering Authority

Organizations authorized to assign CVE IDs.

MITRE, CERT-In, GitHub, and major vendors like Microsoft are CNAs.

To get your own CVE — report a vulnerability to a CNA with a working PoC.

Audit complete — June 6–7, 2026

Phase 1 (Recon): Clean. No secrets in git history.

Phase 2 (Automated): 6 code findings, 64 dependency CVEs.

Phase 3 (Manual): All 6 checks clean. CSRF fixed. One false positive closed.

Net result: 2 open critical findings (same line), 2 open medium findings, 64 dependency CVEs pending upgrades.

Fix Finding #1+#2 first — it's one line change and eliminates RCE.

Then upgrades. Then Dockerfile. Then Jinja2 verification.

Once done — update README and this becomes a portfolio asset.